

Maze Game Project Report

Python / Pygame Maze Generator with BFS Path Finding

Prepared from the uploaded project files: main.py, maze.py, and button.py

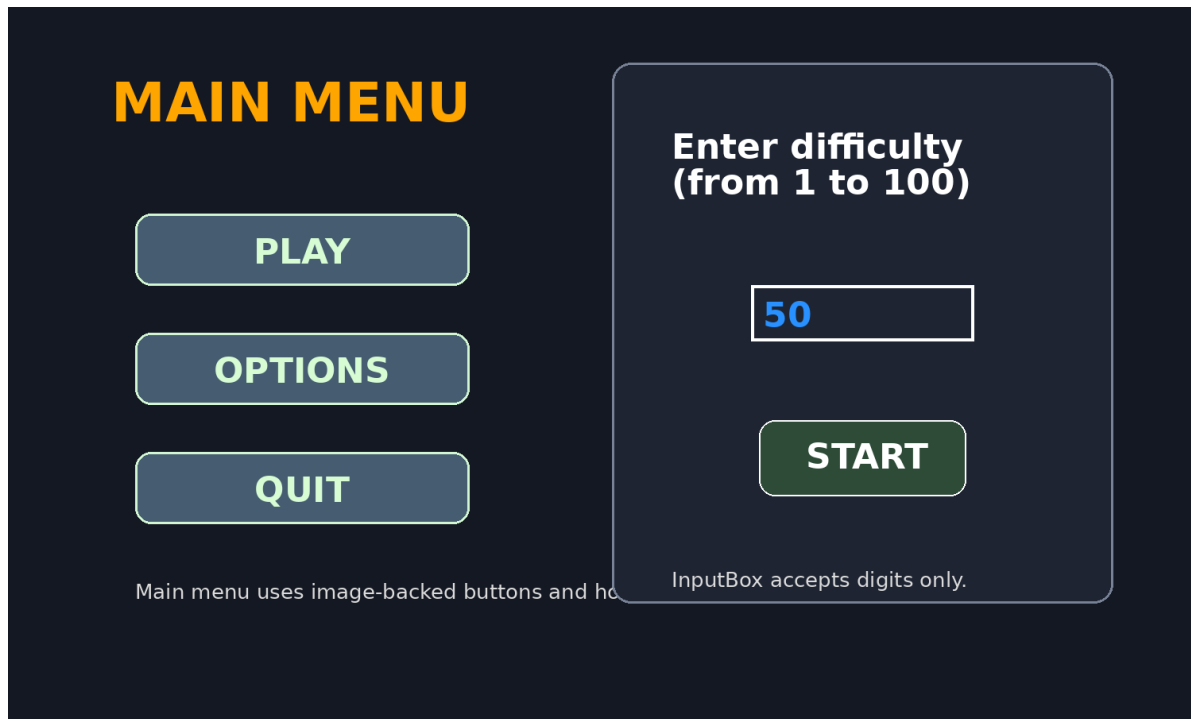


Figure 1. Visual mockup of the main menu and difficulty input screen.

1. Project Overview

This project is a 2D maze game built with Python and Pygame. The player starts from a main menu, chooses a difficulty level from 1 to 100, and the program generates a random 10 x 10 maze. The game then displays the maze using colored cells and uses Breadth-First Search (BFS) to detect and draw a possible route between the start point and the end point.

The project is divided into three main Python files. `main.py` controls the game loop, screens, input boxes, audio controls, and drawing. `maze.py` contains the maze generation logic and BFS path-finding algorithm. `button.py` defines a reusable button class used by the menu screens.

2. Project Objectives

- Create an interactive Pygame application with a main menu, options screen, and maze play screen.
- Allow the user to enter a numeric difficulty level between 1 and 100.
- Generate a random maze where difficulty controls the probability of walls.
- Use BFS to search for a route between the start cell and the end cell.
- Display the generated maze visually using different colors for walls, open cells, start, end, and path.
- Provide basic audio controls such as volume up, volume down, and mute.

3. Source Files and Responsibilities

File	Lines	Main Responsibility	Important Classes / Functions
<code>main.py</code>	226	Runs the Pygame application, menu navigation, input handling, drawing, and audio options.	<code>InputBox</code> , <code>main_menu()</code> , <code>maze_menu()</code> , <code>play()</code> , <code>options()</code> , <code>get_font()</code>
<code>maze.py</code>	53	Builds the random maze, chooses start/end cells, and finds a path using BFS.	<code>generate_maze()</code> , <code>bfs()</code> , <code>draw_path()</code>
<code>button.py</code>	29	Defines reusable interactive menu buttons with hover color changes and click detection.	<code>Button</code> , <code>update()</code> , <code>checkForInput()</code> , <code>changeColor()</code>

In total, the uploaded project contains 308 lines of Python code. Most of the code is in `main.py` because it includes the Pygame windows, loops, event handling, buttons, and audio controls.

4. Project Architecture

Project Structure

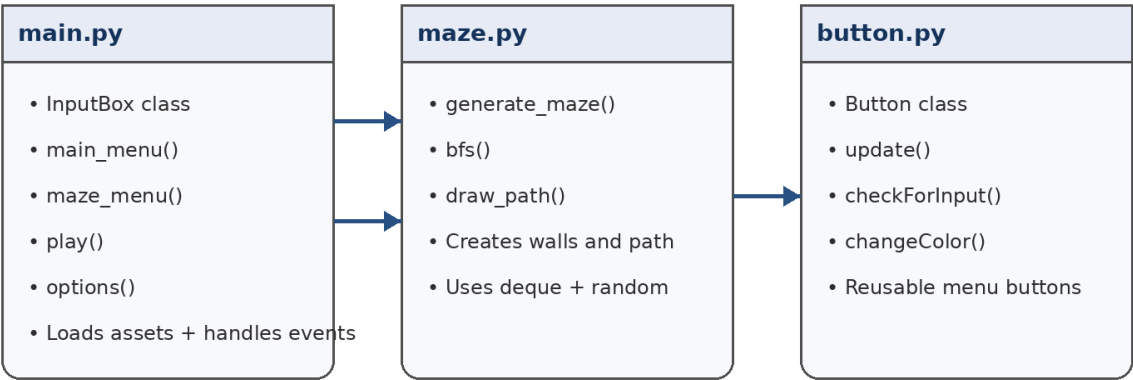
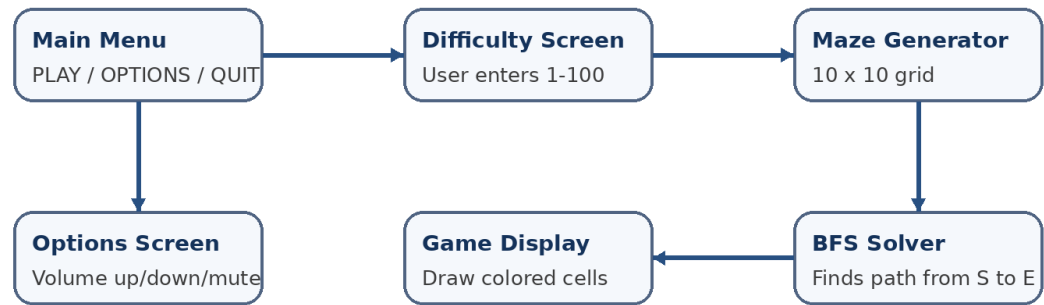


Figure 2. Module structure and responsibility split between the three Python files.

Maze Game Project Flow



The game uses Pygame for screens/buttons and a BFS algorithm for path detection.

Figure 3. Main gameplay flow from menu selection to maze generation and display.

5. User Interface Details

Feature	Implementation Detail	Numeric / Visual Detail
Main window	Created using <code>pygame.display.set_mode()</code> .	Main screen size: 1678 x 944 pixels.
Difficulty screen	Separate menu screen with an InputBox and START button.	Difficulty input range: 1 to 100.

Transition effect	Brief black screen before gameplay starts.	Delay: 300 milliseconds.
Maze drawing	Each maze cell is drawn as a colored rectangle.	Cell size: 60 x 60 pixels.
Maze position	Maze is shifted away from the top-left corner.	Offset: x = 550, y = 100.
Options screen	Buttons for volume up, volume down, mute, and back.	Volume changes by 0.1 per click.

6. Maze Generation and BFS Algorithm

The maze generator receives three values: number of rows, number of columns, and wall probability. In the current game, main.py calls generate_maze(10, 10, difficulty), so each generated maze contains 100 cells.

Symbol / Cell	Meaning	Color in Game
#	Wall / blocked cell	Dark gray
.	Open walkable cell	Light gray
S	Start cell	Green
E	End cell	Red
*	BFS path cell	Cyan

The difficulty number is converted into a wall probability by dividing it by 100. For example, difficulty 30 means each cell has approximately a 30% chance of becoming a wall. BFS then explores neighboring open cells in four directions: down, up, right, and left. When the end cell is found, the program uses the parent dictionary to trace the path backward and mark the solution path with * symbols.

7. Important Numbers in the Project

Item	Value	Explanation
Maze rows	10	The game always generates 10 rows.
Maze columns	10	The game always generates 10 columns.
Total maze cells	100	10 x 10 grid.
Difficulty range	1-100	Accepted input range in the difficulty menu.
Wall probability	difficulty / 100	A difficulty of 50 means roughly 50% walls.
Cell size	60 pixels	Each cell is drawn as a 60 x 60 rectangle minus 2 pixels spacing.
Gameplay maze area	600 x 600 pixels	10 cells x 60 pixels.
Main screen size	1678 x 944 pixels	Main menu and play screen size.
Difficulty screen size	1678 x 1000 pixels	Screen used in maze_menu().
Volume step	0.1	Volume up/down changes sound by 10%.
Transition delay	300 ms	Short black screen before starting the maze.

8. Visual Results

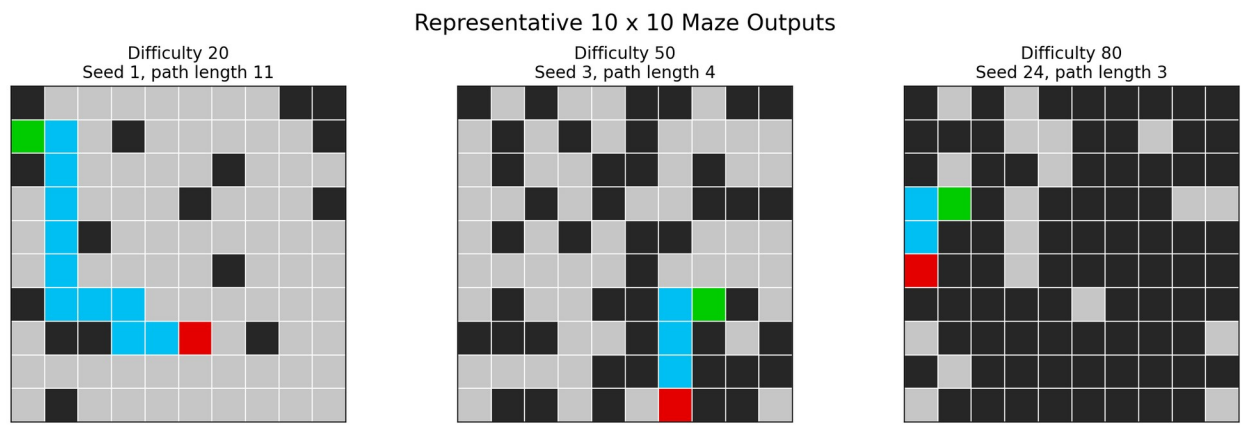


Figure 4. Example generated mazes at different difficulty levels. Dark cells are walls; cyan cells show the BFS route.

9. Difficulty Simulation Results

To understand how the difficulty value affects the maze, a simulation was run using the same 10 x 10 grid idea. Each difficulty level was tested 500 times. The table below shows how often a valid path was found from start to end. Because walls are random, the exact result changes every time the game is run, but the trend is clear: higher difficulty produces more walls and fewer possible paths.

Difficulty	Average Open Cells	Valid Start/End Rate	Path Found Rate	Average Path Length
10	90.0 / 100	100.0%	99.8%	6.9 steps
30	70.0 / 100	100.0%	82.4%	7.6 steps
50	50.1 / 100	100.0%	27.6%	5.3 steps
70	30.0 / 100	100.0%	10.2%	2.9 steps
90	9.9 / 100	99.8%	5.2%	1.5 steps

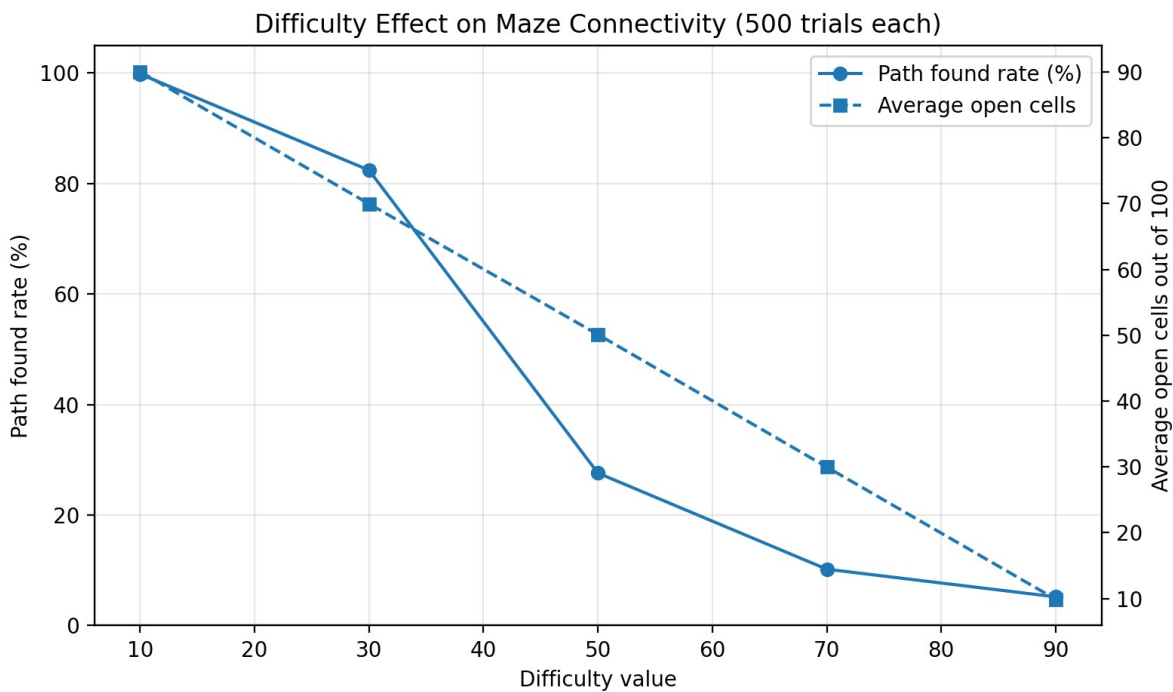


Figure 5. Difficulty increases wall density and lowers the chance of finding a connected path.

10. Strengths of the Project

- The code separates major responsibilities into different files, which makes the project easier to understand.
- The Button class is reusable across the main menu, options menu, and back buttons.
- The InputBox accepts only digits, reducing invalid text input issues.
- The BFS algorithm is an appropriate choice for finding the shortest path in an unweighted grid.
- The game includes visual feedback through different colors for each maze cell type.
- The options screen adds extra polish by allowing volume control.

11. Limitations and Suggested Improvements

Issue	Why it matters	Suggested improvement
Difficulty 100 can freeze maze generation	At 100% wall probability, every cell becomes a wall, so the loop that searches for a non-wall start and end may never finish.	Limit difficulty to 1-99, or force at least two open cells before choosing start and end.
Some mazes may have no path	BFS may fail if walls block the route, and the game still displays the maze without a solution path.	Regenerate the maze until a valid path exists, or show a clear No Path Found message.
Music is played inside the main menu loop	Calling play() repeatedly every frame can restart or overlap audio.	Start music once before the loop or check whether the sound is already playing.
Assets are required but not included in the uploaded files	Images, font, and audio files are needed for the game to run exactly as written.	Include all assets in the project folder or add error handling for missing files.
Button objects are recreated every loop	Creating buttons repeatedly is unnecessary and can reduce efficiency.	Create static buttons once before the loop and update them inside the loop.
Recursive screen switching	Calling main_menu() from inside play() or options() can stack loops.	Use a state variable such as current_screen instead of calling menus recursively.

12. Testing Checklist

Test Case	Expected Result
Click PLAY in main menu	Difficulty screen opens.
Enter a number from 1 to 100 and click START	Game draws a 10 x 10 maze.
Enter letters in difficulty input	Letters should not appear because the InputBox accepts digits only.
Enter an empty value or invalid value	Program prints Invalid difficulty input.
Click VOLUME UP	Sound volume increases by 0.1 until it reaches 1.0.
Click VOLUME DOWN	Sound volume decreases by 0.1 until it reaches 0.0.
Click MUTE	Sound volume becomes 0.0.
Click BACK during gameplay	The program returns to the main menu.
Click QUIT	The application exits.

13. Conclusion

Overall, this project demonstrates a functional Pygame maze application with menus, difficulty input, audio controls, random maze generation, and BFS path finding. The strongest technical part is the use of BFS to search through the grid and mark a path. The project can be improved by adding stronger validation for high difficulty values, preventing no-path mazes, avoiding repeated audio playback, and organizing screen navigation with a cleaner state-based structure.

Appendix: Key Code Logic Summary

Maze generation summary:

1. Convert difficulty into wall probability.
2. Fill each grid cell randomly as wall or open space.
3. Choose random non-wall start and end cells.
4. Run BFS from start to end.
5. Mark the discovered path with * symbols.

Color mapping used during drawing:

= wall, . = open cell, S = start, E = end, * = BFS path